

1 Solution — GIAM 1.5

Problem 1

1.1 Problem

Trace through the division algorithm with inputs $n = 27$ and $d = 5$. Each time an assignment statement is encountered, write it out. How many assignments are involved in this particular computation?

1.2 The Algorithm

The Section 1.5 pseudocode for *Division*:

```
Let  $q = 0$ .  
Let  $r = n$ .  
Label 1.  
If  $r < d$  then Return  $q, r$ .  
Let  $q = q + 1$ .  
Let  $r = r - d$ .  
Goto 1.
```

The four “Let” lines are the **assignment** statements; the If test, Label, Return, and Goto are *not* assignments. So we count every time a “Let” executes.

1.3 The Trace ($n = 27, d = 5$)

Setup (2 assignments):

$$q = 0, \quad r = 27.$$

Test: $27 < 5$? No — enter the loop. Now subtract 5 repeatedly, each pass doing *two* assignments ($q = q + 1$, then $r = r - 5$):

Pass	$q = q + 1$	$r = r - 5$	test $r < 5$?
------	-------------	-------------	----------------

1	$q = 1$	$r = 22$	$22 < 5?$	No
2	$q = 2$	$r = 17$	$17 < 5?$	No
3	$q = 3$	$r = 12$	$12 < 5?$	No
4	$q = 4$	$r = 7$	$7 < 5?$	No
5	$q = 5$	$r = 2$	$2 < 5?$	Yes $\rightarrow$ Return

Table 1.1

The loop runs **5** times; on the fifth, r drops to $2 < 5$, so the algorithm returns

$$q = 5, \quad r = 2, \quad \text{i.e.} \quad 27 = 5 \cdot 5 + 2.$$

1.4 Counting the Assignments

$$\underbrace{2}_{\text{setup: } q=0, r=27} + \underbrace{5 \times 2}_{5 \text{ loop passes, 2 each}} = \boxed{12 \text{ assignments.}}$$

The twelve assignments, in order: $q=0$, $r=27$, $q=1$, $r=22$, $q=2$, $r=17$, $q=3$, $r=12$, $q=4$, $r=7$, $q=5$, $r=2$.

1.5 The Full Trace Table

Tracing Division with $n = 27$, $d = 5$: each "Let ..." is one assignment. There are 12 of them.

#	assignment	q	r	test $r < 5$?
1	Let $q = 0$	0	—	
2	Let $r = 27$	0	27	27 < 5? No → loop
3	Let $q = q+1 \rightarrow 1$	1	27	
4	Let $r = r-5 \rightarrow 22$	1	22	22 < 5? No → loop
5	Let $q = q+1 \rightarrow 2$	2	22	
6	Let $r = r-5 \rightarrow 17$	2	17	17 < 5? No → loop
7	Let $q = q+1 \rightarrow 3$	3	17	
8	Let $r = r-5 \rightarrow 12$	3	12	12 < 5? No → loop
9	Let $q = q+1 \rightarrow 4$	4	12	
10	Let $r = r-5 \rightarrow 7$	4	7	7 < 5? No → loop
11	Let $q = q+1 \rightarrow 5$	5	7	
12	Let $r = r-5 \rightarrow 2$	5	2	2 < 5? Yes → return

12 assignments. Return $q = 5$, $r = 2$: $27 = 5 \cdot 5 + 2$.

count = 2 (setup) + 5 iterations $\times$ 2 = $2 + 2 \cdot \lfloor 27/5 \rfloor = 12$.

Figure 1.1: The 12 assignments in order: setup $q=0$, $r=27$, then five passes each doing $q=q+1$ and $r=r-5$. The running values q and r and the test ' $r < 5$?' are shown at each step. The fifth pass leaves $r=2 < 5$, returning $q=5$, $r=2$ — that is, $27 = 5 \cdot 5 + 2$.

1.6 Remark — The Count Formula $2 + 2\lfloor n/d \rfloor$

The number of assignments is determined by how many times the loop runs, and that count is exactly the quotient. Each pass increments q by 1, and the loop stops when $r < d$ — i.e. after $q = \lfloor n/d \rfloor$ passes. So in general

$$\text{assignments} = \underbrace{2}_{\text{setup}} + 2 \left\lfloor \frac{n}{d} \right\rfloor.$$

For $n = 27$, $d = 5$: $\lfloor 27/5 \rfloor = 5$, giving $2 + 2 \cdot 5 = 12$. ✓ The returned values are precisely $q = \lfloor n/d \rfloor = 5$ and $r = n \bmod d = 2$ — the very quantities of Problems 13–14, now produced *mechanically* by repeated subtraction.

1.7 Remark — The Algorithm Is Repeated Subtraction

This “algorithm” is the most literal possible reading of “division as repeated subtraction”: start with n , subtract d until you can’t anymore, and count how many times you subtracted (q) and what’s left over (r). The textbook itself flags it as “unusually inefficient” — and it is: the number of steps grows *linearly* in the quotient $\lfloor n/d \rfloor$. Dividing a billion by 2 this way would take half a billion subtractions, whereas grade-school long division (or the repeated-division of Problem 1.4.4) handles it in a handful of digit operations. The point of the pseudocode is *not* speed but to show division is achievable by purely mechanical means — a few assignments, a test, and a loop.

1.8 Remark — Why the Tests and Goto Don’t Count

The problem asks specifically for *assignment* statements, so only the “Let” lines are tallied — the 12 above. The loop also executes the **If** $r < d$ test *six* times (once after setup, once after each of the five passes) and the **Goto** five times, but these merely *direct* the flow; they store no new value into a variable. Distinguishing “assignments” (which change state) from “control flow” (which chooses what to do next) is a basic but important distinction in reading algorithms — and it is exactly what this exercise is drilling.

1.9 Remark — A More Efficient Division Would Assign Far Less

Because the assignment count is $2 + 2\lfloor n/d \rfloor$, this algorithm’s cost is governed entirely by the *size of the quotient*. A smarter algorithm subtracts large multiples of d at once (as long division does, one place value at a time), turning the linear $\lfloor n/d \rfloor$ into something logarithmic in n . For our small inputs the difference is invisible — 12 assignments is trivial — but the exercise quietly previews why *algorithmic efficiency*, not just correctness, becomes a central concern once inputs get large. Here, though, the clean takeaway is simply: **12 assignments, yielding $q = 5$ and $r = 2$.**